MAIE 5532: Machine Learning Systems

Week 8 - Automating ML Compilation

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lefteris NTAFLOS, PhD

The Hong Kong University of Science and Technology

Week 8 Overview

Automating ML Compilation: From Models to Efficient Execution

Learning Objectives:

- Understand ML compilation pipelines and their stages
- Master automated optimization techniques for resource-constrained devices
- Analyze cross-platform compilation challenges and solutions
- Design compilation strategies for TensorFlow Lite deployment

Required Readings:

- Book 1 – Volume 1: ML Frameworks
- Book 1 – Volume 1: Hardware Acceleration
- Book 1 – Volume 1: Model Compression
- Book 1 – Volume 1: Model Serving
- Book 2: Chapter 13: TensorFlow Lite for Microcontrollers
- Book 2: Chapter 19: Porting Models from TensorFlow to TensorFlow Lite



Session 1
ML Compilation Fundamentals

- Why ML compilers differ
- The five-stage compilation pipeline
- Graph optimization
- Kernel selection

The Compilation Challenge

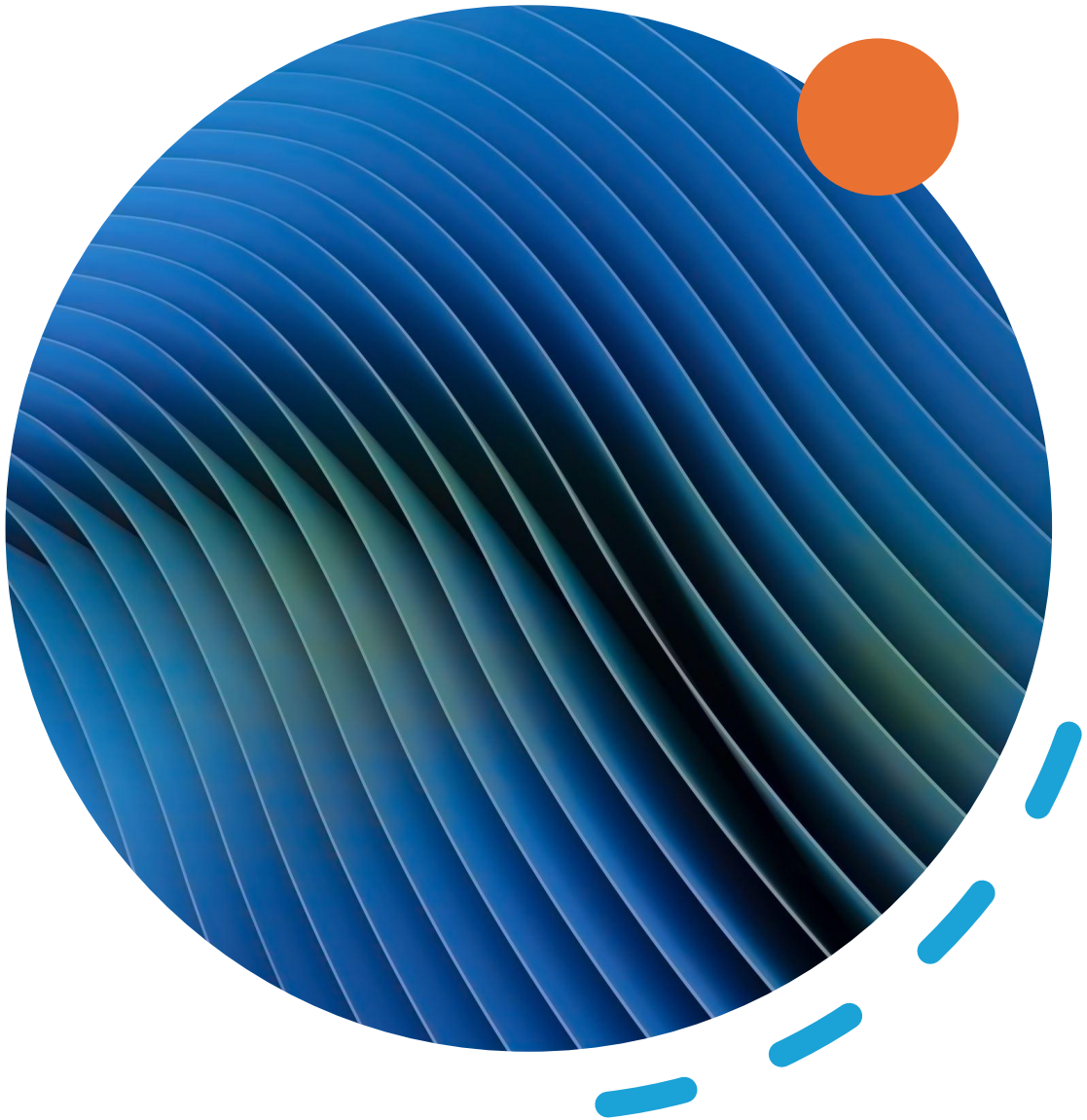
ML models are computation graphs of heavy tensor ops, not linear code

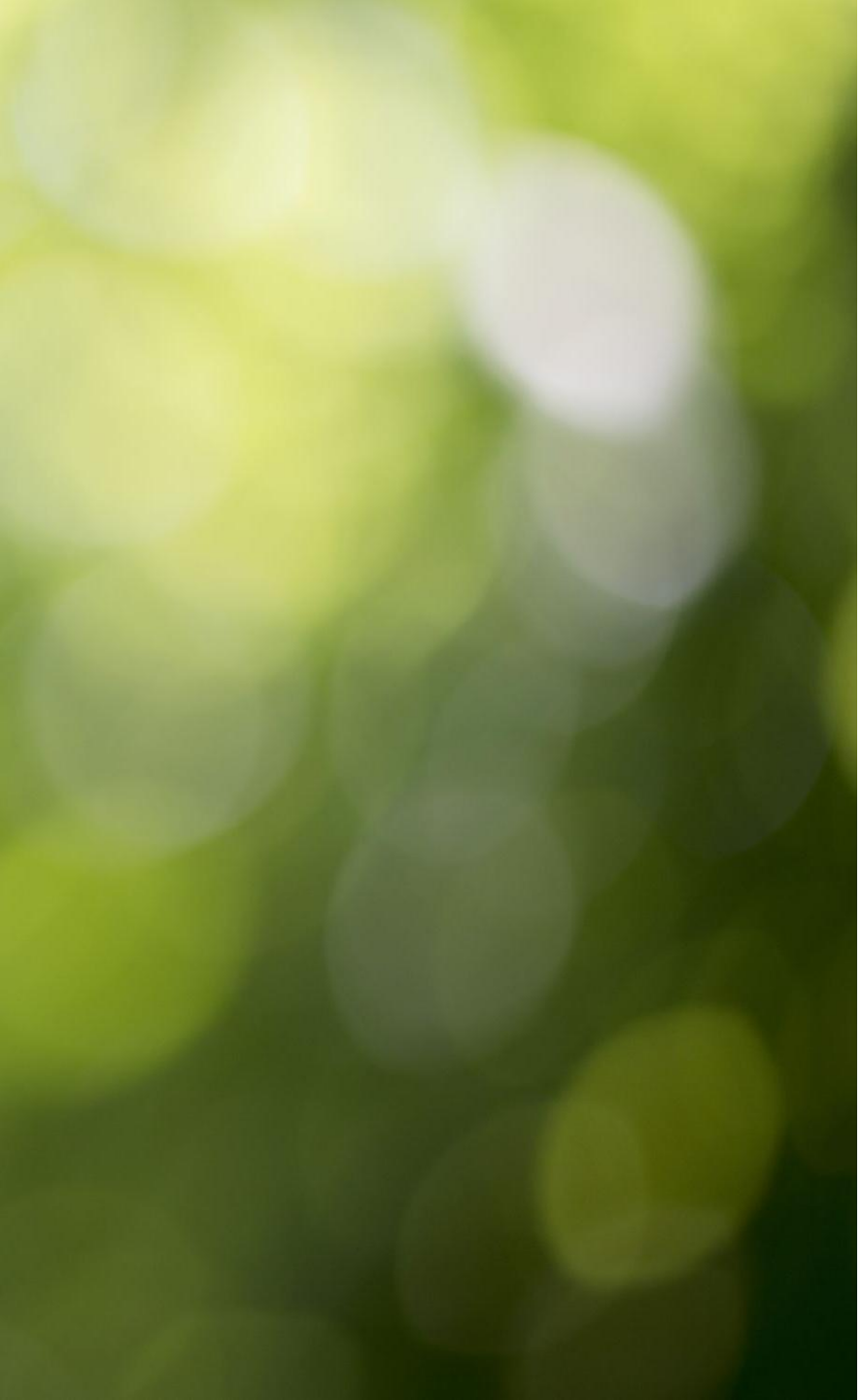
Performance is limited by data movement; locality and reuse beat raw FLOPs

Traditional compilers miss cross-op opportunities across the graph

Need hardware-aware scheduling plus global memory and layout planning

Without ML-specific compilers: stalls, redundant work, low utilization, higher cost





Traditional vs ML Compilers: Big Picture

Traditional: linear code, instruction-level tweaks

- Focuses on per-function optimizations like register allocation, instruction scheduling, and loop transforms to emit fast CPU machine code.

ML: graph-level transformations, tensor-aware scheduling

- Operates on whole computation graphs to fuse ops, align tensor layouts, and choose tiling/parallelism that matches GPUs/TPUs.

Outcome differs: machine code vs accelerator execution plans

- Traditional emits optimized binaries; ML compilers emit fused kernels plus a coordinated plan for memory, launches, and data movement across accelerators.

Traditional vs ML Compilers: Key Differences

Input	Code vs computational graphs
Execution	Sequential/multi-threaded vs massively parallel
Optimization	Registers/scheduling vs fusion/layout/tiling
Memory	Stack/heap vs tensor layouts, buffer reuse
Target	CPUs vs GPUs/TPUs/accelerators

The Five-Stage ML Compilation Pipeline

From High-Level Models to Optimized Execution

1. Graph Optimization — fuse ops, fold constants, delete dead code

2. Kernel Selection — pick a concrete implementation per op (cuDNN, oneDNN, or generated)

3. Memory Planning — layouts, buffer reuse, tiling for on-chip memory

4. Computation Scheduling — when/where work runs; overlap compute with data movement

5. Code Generation — machine code (PTX/cubin, ELF) or an execution plan

A funnel: broad, device-independent rewrites first → concrete device decisions last

Stage 1: Graph Optimization — Goals

- **Improve locality, cut memory traffic** — fuse producer→consumer chains so intermediates never touch DRAM
  - Example: x = a*b; y = x+c; z = relu(y) → one kernel, zero intermediate tensors
- **Remove redundant work** — constant folding, dead code, common subexpressions, algebraic simplification
  - Example: transpose(transpose(x)) → x; inference BatchNorm folded into conv weights
- **Restructure for parallelism** — batch tiny ops, split independent subgraphs, normalize broadcasting
  - Example: Q, K, V projections → one concatenated matmul
- **Stay (mostly) hardware-agnostic** — semantics-preserving rewrites; defer tile sizes and launch configs

Stage 1: Graph Optimization - Implementations

- **XLA (JAX, TPU TensorFlow)**: aggressive fusion + layout assignment on the HLO graph
- **TVM Relay/Relax**: fusion, constant folding, layout rewriting; microTVM brings it to Cortex-M (TinyML)
- **TensorRT (NVIDIA inference)**: Conv+Bias+ReLU fusion, branch merging, dead-layer removal
- **MLIR**: infrastructure, not a compiler — tensor/linalg → bufferization → gpu/llvm dialects (IREE, OpenXLA, Triton build on it)
- **torch.compile / Inductor**: TorchDynamo captures the graph, Inductor fuses and emits Triton kernels

Stage 1: Why Graph Optimization Matters

- **Without it**: one kernel per op → intermediates spill to DRAM, launch overhead dominates, compute idles
  - Unfused multiply→add→relu on 4 MB tensors = 16 MB of avoidable traffic + 2 extra launches
- **With it**: fewer, fatter kernels → data stays on-chip, more independent work, lower latency
  - GELU unfused ≈ 8 element-wise kernels; fused = 1
- **Real-time inference on constrained devices depends on this stage**

LIVE DEMO A — Watching XLA Fuse a Graph

- Open ml_compiler_demos.ipynb on Google Colab
- Runtime → Change runtime type → Hardware accelerator: T4 GPU → Save



Operations and compilation

- ~13 distinct operations
- jit_compile=True
  - XLA compliler

DEMO A — Watching XLA fuse a graph (Stage 1)

Same function, compiled two ways. Then we read the compiler's intermediate representation (HLO) before and after optimization.

A1

[16] ✓ 0s

```
# @title A1
import timeit

tf.random.set_seed(0)
x = tf.random.normal([1024, 1024])
w = tf.random.normal([1024, 1024])
b = tf.random.normal([1024])

def block(x, w, b):
    y = tf.matmul(x, w) + b          # matmul + bias   -> Stage 2 picks the kernel
    y = tf.nn.gelu(y)                # GELU = ~8 element-wise ops (erf-based)
    y = y * 0.5 + 1.0                # 2 more element-wise ops
    return tf.reduce_sum(y, axis=1)  # reduction

graph_fn = tf.function(block)                    # TF graph executor, one kernel per op
xla_fn   = tf.function(block, jit_compile=True)  # XLA compiles the whole function

graph_fn(x, w, b); xla_fn(x, w, b)               # warm-up / compile

n = 100
t_graph = 1000 * timeit.timeit(lambda: graph_fn(x, w, b).numpy(), number=n) / n
t_xla   = 1000 * timeit.timeit(lambda: xla_fn(x, w, b).numpy(),   number=n) / n
print(f"no XLA (one kernel per op) : {t_graph:6.2f} ms")
print(f"XLA (fused)                : {t_xla:6.2f} ms   -> {t_graph/t_xla:.1f}x")
```

```
no XLA (one kernel per op) :   1.80 ms
XLA (fused)                :   1.32 ms   -> 1.4x
```

Number of Instructions and Fusion

- Before and after optimization
- Fusion instructions

A2

[17]
✓ 0s

```
# @title A2
ir     = xla_fn.experimental_get_compiler_ir(x, w, b)
before = ir(stage="hlo")            # HLO straight from TF, before XLA passes
after  = ir(stage="optimized_hlo")  # after fusion, simplification, layout assignment

def n_instr(hlo):
    return sum(1 for line in hlo.splitlines() if " = " in line and "ENTRY" not in line)

print("HLO instructions BEFORE optimization :", n_instr(before))
print("HLO instructions AFTER  optimization :", n_instr(after))
print("  of which 'fusion' instructions     :", sum(1 for l in after.splitlines() if " fusion(" in l))
print("  matmul handed to a library kernel  :", "custom-call" in after or " dot(" in after)
```

```
HLO instructions BEFORE optimization : 119
HLO instructions AFTER  optimization : 100
  of which 'fusion' instructions     : 1
  matmul handed to a library kernel  : True
```

Top-level program after optimization

- **`fusion`** (one kernel launch that replaced many)
- **`custom-call`** (XLA decided not to generate its own matmul code; it hands it to cuBLAS)

A3

```
# @title A3
# Print only the ENTRY computation (the top-level program) so it fits on screen
entry = after[after.index("ENTRY"):]
print(entry)
```

```
ENTRY %a_inference_block_335__.2 (arg0.1: f32[1024,1024], arg1.1: f32[1024,1024], arg2.1: f32[1024]) -> f32[1024] {
  %arg2.1 = f32[1024]{0} parameter(2), parameter_replication={false}, metadata={op_name="XLA_Args"}
  %arg1.1 = f32[1024,1024]{1,0} parameter(1), parameter_replication={false}, metadata={op_name="XLA_Args"}
  %arg0.1 = f32[1024,1024]{1,0} parameter(0), parameter_replication={false}, metadata={op_name="XLA_Args"}
  %custom-call.1 = (f32[1024,1024]{1,0}, s8[4194304]{0}) custom-call(%arg0.1, %arg1.1), custom_call_target="__cublas$gemm",
  %get-tuple-element.1 = f32[1024,1024]{1,0} get-tuple-element(%custom-call.1), index=0, frontend_attributes={grad_x="false"
  ROOT %input_reduce_fusion = f32[1024]{0} fusion(%get-tuple-element.1, %arg2.1), kind=kInput, calls=%fused_reduce, metadata
}
```

fusion

custom-call

Inside one fused kernel

- Read the ops
  - `broadcast, add` (bias)
  - `multiply, erf, add,`(GELU)
- All ops in one kernel
  - **Unfused version**: each line writes to a 4MB tensor to GPU memory, and the next line reads it back
  - **Fused version**: each element is loaded once and is written once

A4

```
# @title A4
# Show the first fused_computation block: this is what used to be ~10 separate kernels
start = after.index("%fused_reduce")
end   = after.index("\n}\n", start) + 3
print(after[start:end])
```

```
%fused_reduce (param_0.129: f32[1024,1024], param_1.174: f32[1024]) -> f32[1024] {
  %mul_6_1 = f32[] constant(0.5), metadata={op_type="Mul" op_name="Gelu/mul" source_file="/usr/local/lib/python3.
  %mul.7.6 = f32[1024,1024]{1,0} broadcast(%mul_6_1), dimensions={}, metadata={op_type="Mul" op_name="Gelu/mul" s
  %param_0.129 = f32[1024,1024]{1,0} parameter(0)
  %param_1.174 = f32[1024]{0} parameter(1)
  %add.3.5 = f32[1024,1024]{1,0} broadcast(%param_1.174), dimensions={1}, metadata={op_type="AddV2" op_name="add"
  %add.4.5 = f32[1024,1024]{1,0} add(%param_0.129, %add.3.5), metadata={op_type="AddV2" op_name="add" source_file
  %mul.8.3 = f32[1024,1024]{1,0} multiply(%mul.7.6, %add.4.5), metadata={op_type="Mul" op_name="Gelu/mul" source_
  %Neg.1.11 = f32[1024,1024]{1,0} negate(%add.4.5), metadata={op_type="Neg" op_name="Gelu/Neg" source_file="/usr/
  %mul_1_3_1 = f32[] constant(0.707106769), metadata={op_type="Mul" op_name="Gelu/mul_1" source_file="/usr/local/
  %mul_1.4.11 = f32[1024,1024]{1,0} broadcast(%mul_1_3_1), dimensions={}, metadata={op_type="Mul" op_name="Gelu/m
  %mul_1.5.11 = f32[1024,1024]{1,0} multiply(%Neg.1.11, %mul_1.4.11), metadata={op_type="Mul" op_name="Gelu/mul_1
  %Erfc.84.8 = f32[1024,1024]{1,0} abs(%mul_1.5.11), metadata={op_type="Erfc" op_name="Gelu/Erfc" source_file="/u
  %Erfc_85_1 = f32[] constant(1), metadata={op_type="Erfc" op_name="Gelu/Erfc" source_file="/usr/local/lib/python
  %Erfc.86.89 = f32[1024,1024]{1,0} broadcast(%Erfc_85_1), dimensions={}, metadata={op_type="Erfc" op_name="Gelu/
  %Erfc.87.3 = pred[1024,1024]{1,0} compare(%Erfc.84.8, %Erfc.86.89), direction=GT, metadata={op_type="Erfc" op_n
  %Erfc_133_1 = f32[] constant(0), metadata={op_type="Erfc" op_name="Gelu/Erfc" source_file="/usr/local/lib/pytho
  %Erfc.134.3 = f32[1024,1024]{1,0} broadcast(%Erfc_133_1), dimensions={}, metadata={op_type="Erfc" op_name="Gelu
  %Erfc.135.3 = pred[1024,1024]{1,0} compare(%mul_1.5.11, %Erfc.134.3), direction=LT, metadata={op_type="Erfc" op
  %Erfc_136_1 = f32[] constant(2), metadata={op_type="Erfc" op_name="Gelu/Erfc" source_file="/usr/local/lib/pytho
  %Erfc.137.5 = f32[1024,1024]{1,0} broadcast(%Erfc_136_1), dimensions={}, metadata={op_type="Erfc" op_name="Gelu
  %Erfc.92.141 = f32[1024,1024]{1,0} multiply(%mul_1.5.11, %mul_1.5.11), metadata={op_type="Erfc" op_name="Gelu/E
  %Erfc.93.15 = f32[1024,1024]{1,0} negate(%Erfc.92.141), metadata={op_type="Erfc" op_name="Gelu/Erfc" source_fil
  %Erfc.94.13 = f32[1024,1024]{1,0} exponential(%Erfc.93.15), metadata={op_type="Erfc" op_name="Gelu/Erfc" source
  %Erfc_98_1 = f32[] constant(0.102081165), metadata={op_type="Erfc" op_name="Gelu/Erfc" source_file="/usr/local/
  %Erfc.99.17 = f32[1024,1024]{1,0} broadcast(%Erfc_98_1), dimensions={}, metadata={op_type="Erfc" op_name="Gelu/
  %Erfc.97.59 = f32[1024,1024]{1,0} divide(%Erfc.86.89, %Erfc.92.141), metadata={op_type="Erfc" op_name="Gelu/Erf
  %Erfc.100.17 = f32[1024,1024]{1,0} multiply(%Erfc.99.17, %Erfc.97.59), metadata={op_type="Erfc" op_name="Gelu/E
```

Stage 2 — Kernel Selection

- Map each (fused) op to a concrete implementation
  - Vendor libraries: cuDNN, cuBLAS/cuBLASLt, oneDNN, Arm Compute Library, Eigen, BLIS
  - Compiler-generated: Triton, TVM, XLA codegen — for unusual shapes or custom fusions
- Precision-aware choices
  - Training: FP32, BF16, FP16 with loss scaling; FP8 on Hopper/Blackwell
  - Inference: FP16/BF16; INT8 or FP8 with calibration when accuracy allows
- A bad choice erases Stage 1's gains — wrong layout forces transposes; wrong tile shape → low occupancy

Stage 2 — Kernel Selection — Strategies

- Rule-based / heuristic — cuDNN heuristics; XLA lowering BF16 matmuls to cuBLAS tensor-core kernels
- Profile-guided (measure on device) — TensorRT times every candidate 'tactic' at engine build; TVM AutoTVM / MetaSchedule
- Cost-model-based (predict) — TVM Ansor learned cost model; XLA GPU analytical model; Halide
- Runtime / lazy — torch.backends.cudnn.benchmark=True; cudnnFind; ONNX Runtime execution providers
- In practice compilers combine all four

Stage 2 — Kernel Selection —Why It Matters

- Good selection: tensor cores / systolic arrays at high occupancy, coalesced access, no extra transposes
- Poor selection: idle compute, extra DRAM traffic, or silent accuracy loss
  - A100 peak: FP32 ≈ 19.5 TFLOPS vs FP16 tensor-core ≈ 312 TFLOPS — a 16× ceiling difference
  - LLM decoding (M=1) with a large-tile GEMM kernel → >90% of the GPU idle
  - Softmax sums / LayerNorm variance in FP16 can overflow — keep accumulations in FP32



Stage 3 — Memory Planning

- Goal: minimize bandwidth and latency; keep data on-chip as long as possible
  - DRAM access ≈ 100× the energy of on-chip SRAM, hundreds of cycles of latency
- **Lever 1** — Tensor layout: NHWC vs NCHW vs tiled/blocked
- **Lever 2** — Access patterns: neighboring threads touch neighboring addresses (coalescing)
- **Lever 3** — Buffer reuse: liveness analysis recycles memory for non-overlapping tensors
- **Lever 4** — Tiling: blocks that fit registers / shared memory / scratchpad



Stage 3 — Memory Planning — Layouts, Reuse, Tiling

- **NVIDIA GPUs**: NHWC for tensor-core convs; blocked NC/32HWC32 for INT8
- **TPUs**: XLA assigns tiled layouts, e.g. T(8,128), matching 8×128 vector registers and the 128×128 MXU (256×256 on v6e+)
  - Dims padded to multiples of 8 and 128 — hidden size 1000 → 1024 (2% waste); 129 → 256 (~50% waste)
- **x86 CPUs (oneDNN)**: blocked nChw16c so one AVX-512 register holds 16 channels
- **Buffer reuse**: lifetimes → interference graph → share regions (same algorithm everywhere)
- **Tiling target**: GPU shared memory, TPU VMEM, CPU L1/L2



Stage 3 — Memory Planning — Why It Matters

- Memory, not arithmetic, is usually the bottleneck
  - A100: ~312 TFLOPS FP16 but only ~2 TB/s HBM → need ~150 FLOPs per byte to keep math units busy
- Without planning: format thrashing, repeated DRAM round-trips, OOMs, jittery latency
- With planning: one layout end-to-end, aggressive reuse, hot data in registers/shared memory
- Bonus: less DRAM traffic = less energy — critical on phones and edge devices

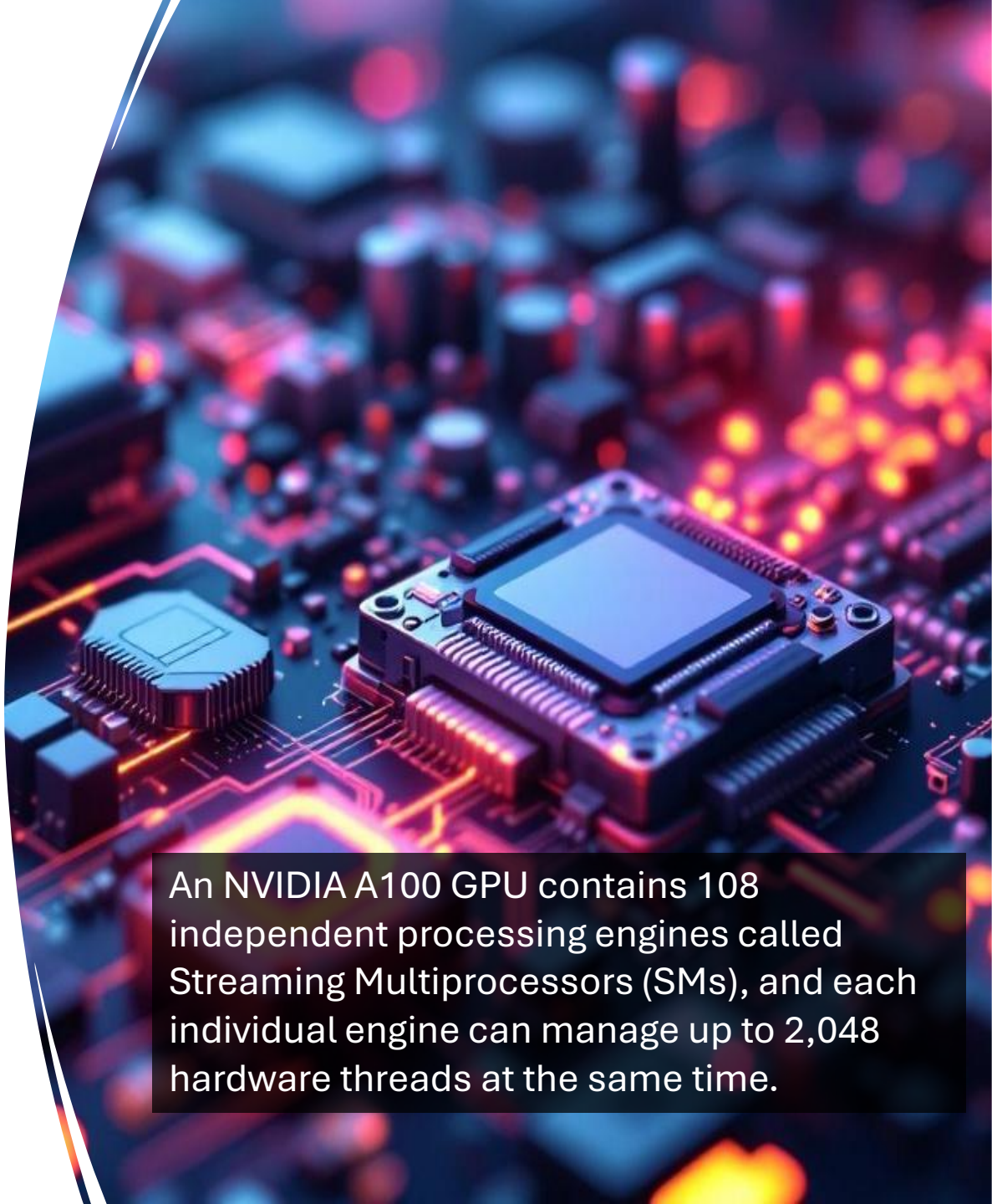
Stage 4 — Computation Scheduling

- Goals: maximize parallelism · keep compute busy · avoid contention · overlap compute with data movement
- Choose tiling and thread/warp/block mappings for high occupancy
- Manage shared memory: avoid bank conflicts, allow blocks to co-reside
- Pipeline loads with compute: double buffering, async copies (cp.async on Ampere, TMA on Hopper)
- Example: FlashAttention is a scheduling result — tile Q/K/V so the N×N score matrix is never materialized



Stage 4 — How Compilers Schedule

- GPUs: tiles → thread blocks → warps; balance occupancy vs register/shared-memory pressure
- TPUs: matmul tiles streamed through the systolic array; XLA statically schedules everything (no dynamic warp scheduler)
- CPUs: block loops for cache, vectorize inner loops (AVX-512 / NEON), spread outer loops across threads, NUMA-aware
- Ordering: topological schedule by readiness; independent ops on separate streams
    - Prefetch next layer's weights or next KV-cache block while current compute runs



An NVIDIA A100 GPU contains 108 independent processing engines called Streaming Multiprocessors (SMs), and each individual engine can manage up to 2,048 hardware threads at the same time.

Stage 4 — Streaming and Synchronization

- Overlap compute and transfers to stay compute-bound
  - Example: host→device copy of batch n+1 on stream 1 while batch n computes (PyTorch: pin_memory=True, non_blocking=True)
- Minimal, precise synchronization — barriers only at true dependencies
  - Example: __syncthreads() once per K-tile, not per element; CUDA events instead of device-wide syncs
- Resource budgeting — shared-memory bank conflicts, register spilling, launch-queue thrash



Stage 5 — Code Generation

- **Classic back-end work**: instruction selection → register allocation → vectorization / tensor-core intrinsics → instruction scheduling & prefetching
- **CPU output**: object code / ELF binary via LLVM
- **GPU output:** PTX (virtual ISA) → cubin for sm_80, sm_90…; SPIR-V for Vulkan/OpenCL; AMD GCN/CDNA ISA
- **Accelerator output (TPU, NPU):** execution plan or bytecode — kernel sequence + DMA descriptors + sync markers
- **Example**: torch.compile → Triton kernel → PTX → cubin, plus launch config grid=(M/128, N/128)

ISA (Instruction Set Architecture): The set of machine instructions a processor understands (e.g., x86-64, ARMv8, NVIDIA PTX as a virtual ISA).

PTX (Parallel Thread Execution): NVIDIA's virtual GPU instruction set used as an intermediate target before device-specific binaries (cubin).

DMA (Direct Memory Access): Hardware-supported data transfer between memories or devices without involving the main CPU cores.

ELF (Executable and Linkable Format): A common binary format for executables and libraries on Unix-like systems.

Cubin (CUDA binary): A device-specific compiled binary containing GPU machine code generated from PTX for a particular NVIDIA GPU architecture (e.g., sm_80).

SPIR-V: cross-vendor intermediate representation for Vulkan/OpenCL GPUs

Triton: Python-like GPU kernel language (used by torch.compile) that compiles to PTX

LIVE DEMO B — torch.compile on a GPU (Google Colab, T4)

- Platform: colab.research.google.com → Open ml_compiler_demos.ipynb → Runtime → Change runtime type → T4 GPU
- Eager vs torch.compile on Conv→BN→ReLU→GELU: count CUDA kernels with torch.profiler (Stage 1)
- torch._logging.set_logs(output_code=True): read the generated Triton kernel (Stage 5)
- Bonus (Stage 2): 4096^2 matmul FP32 vs FP16 tensor cores — expect ~4–6× on a T4

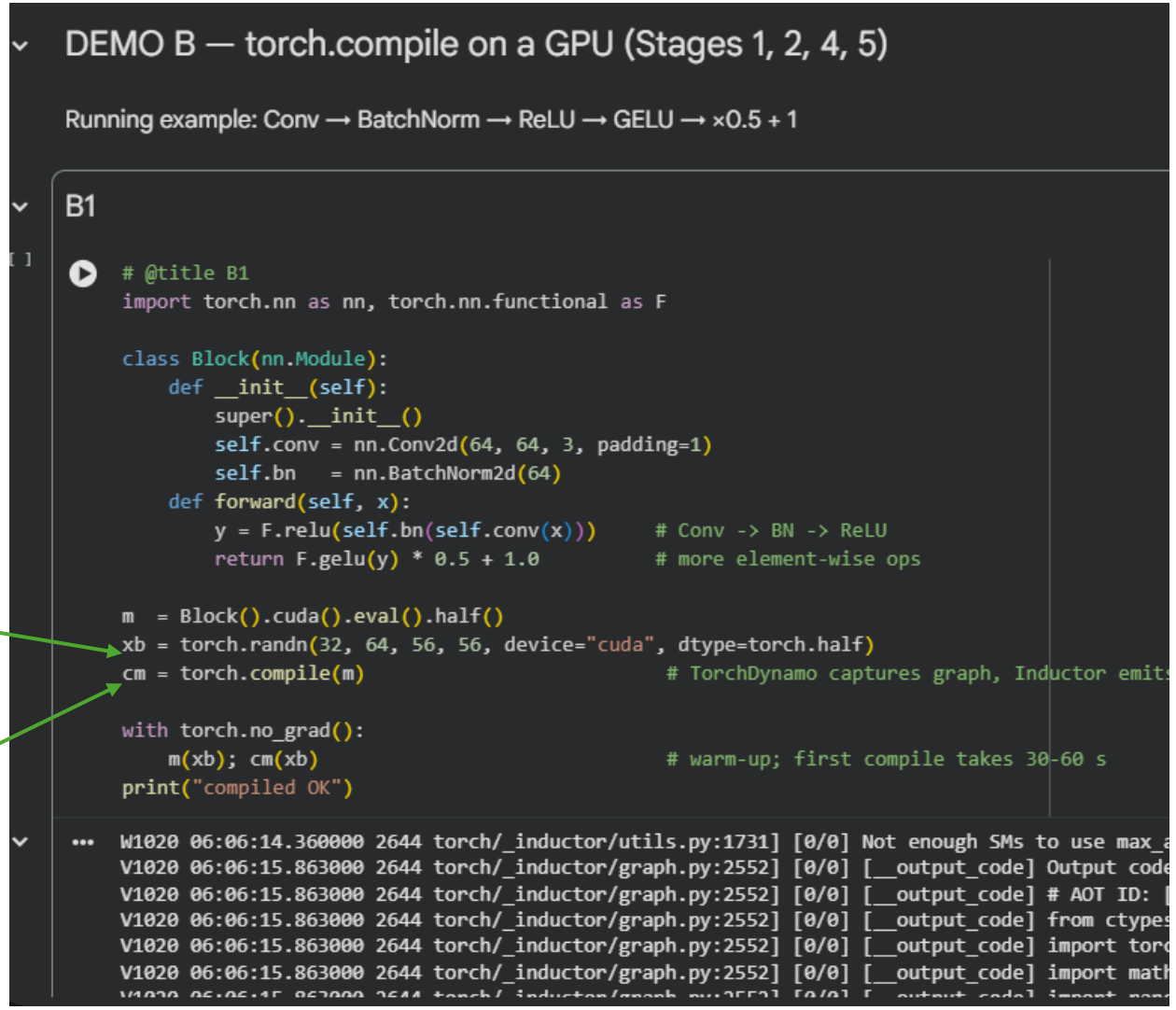


Eager / compiled execution

- TorchDynamo captures the Python into a graph, and Inductor — the back end — fuses it and writes Triton kernels for the GPU."

eager

torch.compile



Kernel Launces

- Eager vs compiled

B2

```python
# @title B2
from torch.profiler import profile, ProfilerActivity

def show(fn, label):
    with torch.no_grad(), profile(activities=[ProfilerActivity.CUDA]) as p:
        fn(xb); torch.cuda.synchronize()
    ka = p.key_averages()
    n  = sum(k.count for k in ka if k.device_type == torch.autograd.DeviceType.CUDA)
    print(f"\n===== {label}: {n} CUDA kernel launches =====")
    print(ka.table(sort_by="cuda_time_total", row_limit=10))

show(m,  "EAGER (one kernel per op)")
show(cm, "torch.compile (fused)")
```

```
===== EAGER (one kernel per op): 12 CUDA kernel launches =====
-------------------------------------------------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------
                                                   Name    Self CPU %     Self CPU   CPU total %     CPU total  CPU time avg     Self CUDA   Self CUDA %    CUDA total  CUDA time avg    # of Calls
-------------------------------------------------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------
sm75_xmma_fprop_implicit_gemm_f16f16_f16f32_f32_nhwc...       0.00%       0.000us       0.00%       0.000us       0.000us     763.601us        39.86%     763.601us     763.601us             1
void at::native::elementwise_kernel<128, 4, at::nati...       0.00%       0.000us       0.00%       0.000us       0.000us     221.275us        11.55%     221.275us     221.275us             1
void at::native::batch_norm_transform_input_kernel<c...       0.00%       0.000us       0.00%       0.000us       0.000us     206.652us        10.79%     206.652us     206.652us             1
void cudnn::engines_precompiled::nchwToNhwcKernel<__...       0.00%       0.000us       0.00%       0.000us       0.000us     155.293us         8.11%     155.293us      77.647us             2
void cudnn::engines_precompiled::nhwcToNchwKernel<__...       0.00%       0.000us       0.00%       0.000us       0.000us     134.365us         7.01%     134.365us     134.365us             1
void at::native::vectorized_elementwise_kernel<4, at...       0.00%       0.000us       0.00%       0.000us       0.000us     108.350us         5.66%     108.350us     108.350us             1
void at::native::vectorized_elementwise_kernel<4, at...       0.00%       0.000us       0.00%       0.000us       0.000us     106.557us         5.56%     106.557us     106.557us             1
void at::native::vectorized_elementwise_kernel<4, at...       0.00%       0.000us       0.00%       0.000us       0.000us     105.854us         5.52%     105.854us     105.854us             1
void at::native::vectorized_elementwise_kernel<4, at...       0.00%       0.000us       0.00%       0.000us       0.000us     104.894us         5.47%     104.894us     104.894us             1
void at::native::unrolled_elementwise_kernel<at::nat...       0.00%       0.000us       0.00%       0.000us       0.000us       4.992us         0.26%       4.992us       4.992us             1
-------------------------------------------------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------
Self CPU time total: 5.060ms
Self CUDA time total: 1.916ms
```

```
===== torch.compile (fused): 4 CUDA kernel launches =====
-------------------------------------------------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------
                                                   Name    Self CPU %     Self CPU   CPU total %     CPU total  CPU time avg     Self CUDA   Self CUDA %    CUDA total  CUDA time avg    # of Calls
-------------------------------------------------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------
sm75_xmma_fprop_implicit_gemm_f16f16_f16f32_f32_nhwc...       0.00%       0.000us       0.00%       0.000us       0.000us     563.286us        69.06%     563.286us     563.286us             1
triton_poi_fused__native_batch_norm_legit_no_trainin...       0.00%       0.000us       0.00%       0.000us       0.000us     140.542us        17.23%     140.542us     140.542us             1
                                triton_poi_fused_convolution_0       0.00%       0.000us       0.00%       0.000us       0.000us     107.710us        13.21%     107.710us     107.710us             1
                                triton_poi_fused_convolution_1       0.00%       0.000us       0.00%       0.000us       0.000us       4.096us         0.50%       4.096us       4.096us             1
                                      Activity Buffer Request      88.02%       4.033ms      88.02%       4.033ms       4.033ms       0.000us         0.00%       0.000us       0.000us             1
                                                  cuLaunchKernel       1.85%      84.667us       1.85%      84.667us      28.222us       0.000us         0.00%       0.000us       0.000us             3
                                               cudaPeekAtLastError       0.05%       2.206us       0.05%       2.206us       1.103us       0.000us         0.00%       0.000us       0.000us             2
                                             cudaLaunchKernelExC       0.76%      34.763us       0.76%      34.763us      34.763us       0.000us         0.00%       0.000us       0.000us             1
                                           cudaDeviceSynchronize       9.32%     427.169us       9.32%     427.169us     213.584us       0.000us         0.00%       0.000us       0.000us             2
-------------------------------------------------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------  ------------
Self CPU time total: 4.582ms
Self CUDA time total: 815.634us
```

Compilation

B3

```
# @title B3
import torch._dynamo
torch._logging.set_logs(output_code=True)      # print Inductor's generated source
torch._dynamo.reset()                          # force a fresh compile so the code prints
cm2 = torch.compile(m)
with torch.no_grad():
    cm2(xb)
torch._logging.set_logs()                      # logging off again
```

```
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]    triton_meta={'signature': {'in_ptr0': '*fp16', 'out_ptr0': '*fp16', 'ynumel': 'i32', 'xnumel': 'i32', 'YBLOCK': 'constexpr', 'XBLOCK':
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]    inductor_meta={'grid_type': 'Grid2D', 'autotune_hints': set(), 'kernel_name': 'triton_poi_fused_convolution_0', 'mutated_arg_names': [
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]    min_elem_per_thread=0
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code] )
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code] @triton.jit
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code] def triton_poi_fused_convolution_0(in_ptr0, out_ptr0, ynumel, xnumel, YBLOCK : tl.constexpr, XBLOCK : tl.constexpr):
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]     ynumel = 2048
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]     xnumel = 3136
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]     yoffset = tl.program_id(1) * YBLOCK
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]     yindex = yoffset + tl.arange(0, YBLOCK)[:, None]
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]     ymask = tl.full([YBLOCK], True, tl.int1)[:, None]
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]     xoffset = tl.program_id(0) * XBLOCK
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]     xindex = xoffset + tl.arange(0, XBLOCK)[None, :]
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]     xmask = xindex < xnumel
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]     x2 = xindex
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]     y3 = yindex
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]     y0 = (yindex % 64)
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]     y1 = yindex // 64
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]     tmp0 = tl.load(in_ptr0 + (x2 + 3136*y3), xmask, eviction_policy='evict_last').to(tl.float32)
V1020 06:06:26.214000 2644 torch/_inductor/codecache.py:1347] [0/0] [__output_code]     tl.store(out_ptr0 + (y0 + 64*x2 + 200704*y1), tmp0, xmask)
```

```
def call(self, args):
    arg0_1, arg1_1, arg2_1, arg3_1, arg4_1, arg5_1, arg6_1 = args
    args.clear()
    assert_size_stride(arg0_1, (64, 64, 3, 3), (576, 9, 3, 1))
    assert_size_stride(arg1_1, (64, ), (1, ))
    assert_size_stride(arg2_1, (32, 64, 56, 56), (200704, 3136, 56, 1))
    assert_size_stride(arg3_1, (64, ), (1, ))
    assert_size_stride(arg4_1, (64, ), (1, ))
    assert_size_stride(arg5_1, (64, ), (1, ))
    assert_size_stride(arg6_1, (64, ), (1, ))
    with torch.cuda._DeviceGuard(0):
        torch.cuda.set_device(0)
        buf0 = empty_strided_cuda((32, 64, 56, 56), (200704, 1, 3584, 64), torch.float16)
        # Topologically Sorted Source Nodes: [conv2d], Original ATen: [aten.convolution]
        stream0 = get_raw_stream(0)
        triton_poi_fused_convolution_0.run(arg2_1, buf0, 2048, 3136, stream=stream0)
        del arg2_1
        buf1 = empty_strided_cuda((64, 64, 3, 3), (576, 1, 192, 64), torch.float16)
        # Topologically Sorted Source Nodes: [conv2d], Original ATen: [aten.convolution]
        stream0 = get_raw_stream(0)
        triton_poi_fused_convolution_1.run(arg0_1, buf1, 4096, 9, stream=stream0)
        del arg0_1
        # Topologically Sorted Source Nodes: [conv2d], Original ATen: [aten.convolution]
        buf2 = extern_kernels.convolution(buf0, buf1, stride=(1, 1), padding=(1, 1), dilation=(1, 1), transposed=False, output_padding=(0, 0), groups=1, bias=None)
        assert_size_stride(buf2, (32, 64, 56, 56), (200704, 1, 3584, 64), 'torch.ops.aten.convolution.default')
        del buf1
        buf4 = reinterpret_tensor(buf0, (32, 64, 56, 56), (200704, 3136, 56, 1), 0); del buf0  # reuse
        # Topologically Sorted Source Nodes: [conv2d, batch_norm, y, gelu, mul, add], Original ATen: [aten.convolution, aten._native_batch_norm_legit_no_training, aten.relu, aten.gelu, aten.mul, aten.add]
        stream0 = get_raw_stream(0)
        triton_poi_fused__native_batch_norm_legit_no_training_add_convolution_gelu_mul_relu_2.run(buf2, arg1_1, arg3_1, arg4_1, arg5_1, arg6_1, buf4, 100352, 64, stream=stream0)
        del arg1_1
        del arg3_1
        del arg4_1
        del arg5_1
        del arg6_1
        del buf2
    return (buf4, )
```

- xnumel = … and XBLOCK: the tile size
- tl.load(…) lines : Loads from global memory
- tmp arithmetic: the fused math
- tl.store(…): One store. Load once, compute everything, store once.
- def call(args): the launch plan

Total TFLOPS

- Eager vs compiled

B4

```python
# @title B4
def bench(fn, n=20):
    fn(); torch.cuda.synchronize()
    s, e = torch.cuda.Event(enable_timing=True), torch.cuda.Event(enable_timing=True)
    s.record()
    for _ in range(n): fn()
    e.record(); torch.cuda.synchronize()
    return s.elapsed_time(e) / n

a  = torch.randn(4096, 4096, device="cuda")
ah = a.half()
t32, t16 = bench(lambda: a @ a), bench(lambda: ah @ ah)
flops = 2 * 4096**3
print(f"FP32 (CUDA cores)   : {t32:6.2f} ms   {flops/t32/1e9:6.1f} TFLOPS")
print(f"FP16 (tensor cores) : {t16:6.2f} ms   {flops/t16/1e9:6.1f} TFLOPS   -> {t32/t16:.1f}x faster")
```

```
FP32 (CUDA cores)   :  41.18 ms      3.3 TFLOPS
FP16 (tensor cores) :   6.78 ms     20.3 TFLOPS   -> 6.1x faster
```

B5

```python
# @title B5
import torch._inductor.config as cfg
cfg.freezing = True                          # inference-only: fold BN constants into conv weights
torch._dynamo.reset()
cm3 = torch.compile(m)
with torch.no_grad(): cm3(xb)
show(cm3, "torch.compile + freezing (BN folded)")
cfg.freezing = False
```

```
===== torch.compile + freezing (BN folded): 3 CUDA kernel launches =====
```

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	Self CUDA	Self CUDA %	CUDA total	CUDA time avg	# of Calls
sm75_xmma_fprop_implicit_gemm_f16f16_f16f32_f32_nhwc...	0.00%	0.000us	0.00%	0.000us	0.000us	564.564us	71.75%	564.564us	564.564us	1
triton_poi_fused_add_convolution_gelu_mul_relu_1	0.00%	0.000us	0.00%	0.000us	0.000us	115.358us	14.66%	115.358us	115.358us	1
triton_poi_fused_convolution_0	0.00%	0.000us	0.00%	0.000us	0.000us	106.877us	13.58%	106.877us	106.877us	1
Activity Buffer Request	78.21%	2.139ms	78.21%	2.139ms	2.139ms	0.000us	0.00%	0.000us	0.000us	1
cuLaunchKernel	2.53%	69.072us	2.53%	69.072us	34.536us	0.000us	0.00%	0.000us	0.000us	2
cudaPeekAtLastError	0.06%	1.767us	0.06%	1.767us	0.884us	0.000us	0.00%	0.000us	0.000us	2
cudaLaunchKernelExC	0.58%	15.750us	0.58%	15.750us	15.750us	0.000us	0.00%	0.000us	0.000us	1
cudaDeviceSynchronize	18.62%	509.187us	18.62%	509.187us	254.594us	0.000us	0.00%	0.000us	0.000us	2

```
Self CPU time total: 2.734ms
Self CUDA time total: 786.799us
```

Compilation + Runtime

- **Why a runtime?** Batch sizes change, shapes are dynamic, devices are shared
- **Memory**: PyTorch caching allocator; vLLM PagedAttention KV-cache pool
- **Kernel dispatch**: pick among precompiled variants per shape — TensorRT optimization profiles, cuDNN benchmark cache
- **Scheduling**: streams, dynamic batching (Triton Inference Server), multi-GPU collectives (NCCL)
- **Exception** — microcontrollers: TFLite Micro / microTVM use a static memory arena and frozen op order; nothing allocated during inference

Quiz 1 – Question 1

Which pipeline stage selects hardware-optimized implementations for each operation?

A) Graph optimization
B) Kernel selection
C) Memory planning
D) Code generation

Quiz 1 – Answer 1

Which pipeline stage selects hardware-optimized implementations for each operation?

A) Graph optimization

B) Kernel selection

C) Memory planning

D) Code generation

It maps each operation to the most efficient hardware-specific kernel

Quiz 1 – Question 2

A primary benefit of kernel fusion is:

A) More kernel launches

B) More intermediate tensors in DRAM

C) Reduced memory traffic by keeping intermediates in fast memory

D) Disabling activations

Quiz 1 – Answer 2

A primary benefit of kernel fusion is:

A) More kernel launches

B) More intermediate tensors in DRAM

C) Reduced memory traffic by keeping intermediates in fast memory

D) Disabling activations

Fusion keeps intermediates in registers/on-chip, reducing memory writes and launches

Quiz 1 – Question 3

Cost model-based kernel selection (MLIR) relies on:

A) Random choice
B) Predicting execution time and memory cost
C) Disabling tiling
D) Eliminating fusion

Quiz 1 – Answer 3

Cost model-based kernel selection (MLIR) relies on:

A) Random choice

B) Predicting execution time and memory cost

C) Disabling tiling

D) Eliminating fusion

Predicts performance and memory to pick minimal-cost kernels

Quiz 1 – Question 4

In memory planning, tensor layout selection:

A) Is unrelated to performance
B) Is always NHWC
C) Aligns storage to hardware-preferred access patterns
D) Requires dynamic allocation

Quiz 1 – Answer 4

In memory planning, tensor layout selection:

A) Is unrelated to performance

B) Is always NHWC

C) Aligns storage to hardware-preferred access patterns

D) Requires dynamic allocation

Choose layouts (e.g., NHWC vs NCHW) to match hardware's access patterns

Quiz 1 – Question 5

Tiling primarily helps by:

A) Keeping working sets in fast memory to reduce bandwidth

B) Increasing off-chip accesses

C) Eliminating the need for scheduling

D) Removing kernel selection

Quiz 1 – Answer 5

Tiling primarily helps by:

A) Keeping working sets in fast memory to reduce bandwidth

B) Increasing off-chip accesses

C) Eliminating the need for scheduling

D) Removing kernel selection

Tiles fit in local memory, cutting off-chip traffic and stalls

Quiz 1 – Question 6

Computation scheduling overlaps compute and transfers to:

A) Increase stalls

B) Disable streaming

C) Force serialization

D) Keep execution compute-bound

Quiz 1 – Answer 6

Computation scheduling overlaps compute and transfers to:

A) Increase stalls

B) Disable streaming

C) Force serialization

D) Keep execution compute-bound

Overlap hides memory latency, keeping compute units busy

Quiz 1 – Question 7

Precision-aware kernel selection is important because:

A) INT8 always preserves accuracy

B) FP32 is always slower

C) Precision impacts both performance and numerical stability

D) Precision doesn’t affect power

Quiz 1 – Answer 7

Precision-aware kernel selection is important because:

A) INT8 always preserves accuracy

B) FP32 is always slower

C) Precision impacts both performance and numerical stability

D) Precision doesn’t affect power

Choose precision to balance speed/power versus accuracy

Quiz 1 – Question 8

Without graph optimization, models often:

A) Underutilize hardware due to redundant ops and poor data locality

B) Remove stale dependencies

C) Avoid redundant writes

D) Automatically reuse caches

Quiz 1 – Answer 8

Without graph optimization, models often:

A) Underutilize hardware due to redundant ops and poor data locality

B) Remove stale dependencies

C) Avoid redundant writes

D) Automatically reuse caches

Redundant ops and memory stalls reduce utilization

Quiz 1 – Question 9

Code generation in ML compilers:

A) Never uses vectorization

B) Is identical to traditional without differences

C) Emits target-specific code/graphs using vectorization/prefetching

D) Skips register allocation

Quiz 1 – Answer 9

Code generation in ML compilers:

A) Never uses vectorization

B) Is identical to traditional without differences

C) Emits target-specific code/graphs using vectorization/prefetching

D) Skips register allocation

It finalizes optimized execution with traditional techniques

Quiz 1 – Question 10

Runtime support complements compilation by:

A) Avoiding dynamic decisions

B) Handling dynamic workload/scheduling/coordination at runtime

C) Disabling streaming

D) Removing codegen

Quiz 1 – Answer 10

Runtime support complements compilation by:

A) Avoiding dynamic decisions

B) Handling dynamic workload/scheduling/coordination at runtime

C) Disabling streaming

D) Removing codegen

Runtimes adapt to real-time conditions beyond static plans

SESSION 2: CROSS-PLATFORM COMPILATION

TF → TFLite: Know Your Ops and TFLite Operation Coverage

- TF → TFLite: know your ops early
  - List the underlying TensorFlow ops—even before training—to check TFLite coverage and plan replacements if needed.
- Keras hides ops inside layers; inspect via graph/session tools
  - Export or trace the model to a concrete graph and iterate nodes to reveal actual ops (Conv2D, Reshape, Gather, etc.).
- TFLite operation coverage is a subset; microcontroller branch is smaller
  - Expect gaps versus TF; TFLite Micro supports even fewer ops and often requires kernels tailored to fixed shapes.
- Export early to catch unsupported ops (e.g., many RNN variants)
  - Trial conversions surface missing kernels, layout quirks, and converter flags you'll need to adjust.
- Training/backprop ops are stripped; floats may be quantized
  - Only inference ops remain; post-training quantization or quantization-aware training can replace FP32 with INT8/FP16 where supported.

Listing Underlying TF Ops

- Goal: enumerate op types used by your model to compare with TFLite coverage
- Keras: ops are inside layers; inspect via session graph
- Example approach:
  - Acquire Session: tf.keras.backend.get_session()
  - Iterate: for op in sess.graph.get_operations(): print(op.type)
- Use output list to map against TFLite/Micro op support and plan refactors

TFLite Op Coverage Mindset

- TFLite $\subset$ TF; TFLite Micro $\subset$ TFLite (micro has fewer ops)
- RNNs (e.g., LSTMs, GRUs) often unsupported in Micro; confirm latest coverage
- Some exports transform graphs (e.g., fusing, quantization, const folding)
- Practical rule: keep core NN simple; avoid exotic/training-centric ops

Export Early, Export Often

- Export an initial TFLite model as soon as the structure exists
- Validate:
  - Unsupported ops present?
  - Float-to-int quantization paths available?
  - Model size and expected memory fit?
- Adjust architecture while still cheap to change

Separate Core NN vs Pre/Post

- Three stages: preprocessing → core NN → postprocessing
- Preprocessing examples:
  - Image: DecodeJpeg → ResizeImages; prefer app-side for fixed camera feed
  - Audio: FFT/mel; cache and reuse for streaming input
- Postprocessing examples:
  - Classification smoothing/averaging over time
  - Detection: non-max suppression (NMS) → app-side for better efficiency
- Benefits: performance, control (threading/cache), maintainability, smaller model

App-Side Preprocessing Patterns

- Fixed sensor pipeline: avoid general-purpose ops in graph
- Camera: direct resize/crop to model’s input dims; no JPEG decode on-device if not needed
- Audio: incremental FFT windows; reuse cached spectra for streaming
- Threading opportunities: decouple capture, prep, inference

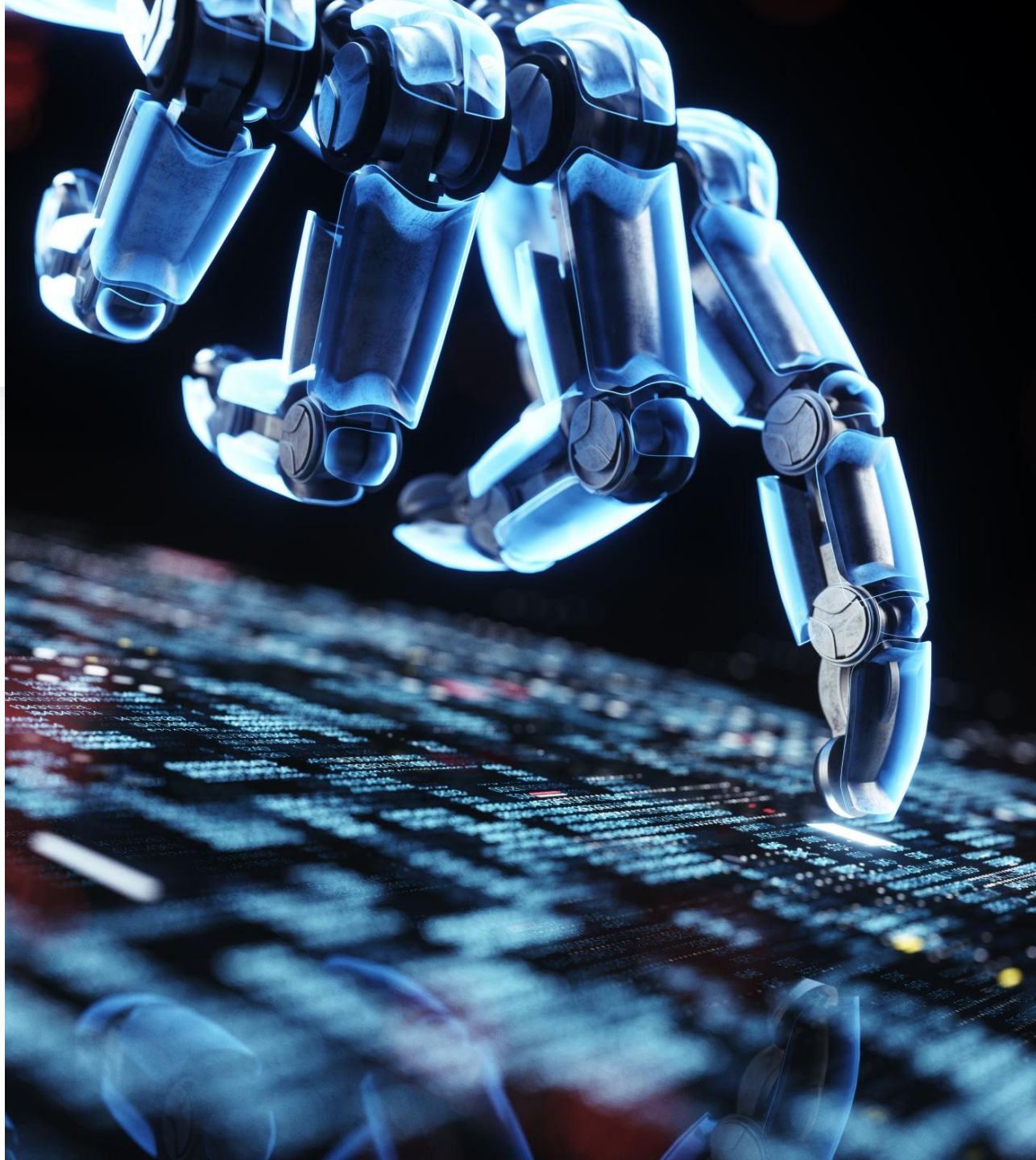


App-Side Postprocessing Patterns

- Streaming classification smoothing: temporal averaging for wake words
- Thresholding/hysteresis: stabilize outputs before UI/actuation
- Detection: NMS external to graph; faster and simpler than graph-level composition
- Device coherence: exploit time locality across repeated inferences

When You Must Use Custom Ops

- Only if redesign is infeasible and op is essential
- Steps:
  - Convert with allow_custom_ops to embed unsupported ops as custom
  - Implement custom kernel; register via AddCustom() in your OpResolver
  - Parse parameters via FlexBuffers in Init()
- Trade-offs: increased dev effort; portability and maintenance burden



Optimize Ops Early

- Supported ops may still be slow for your use case
- Profile latency/footprint with untrained or minimally trained model
- If bottleneck identified, plan:
  - Architectural change (simpler op)
  - Quantization/int8 path
  - Kernel-level optimization (Micro: portable_optimized/)
- Avoid “late surprises” near deployment

Quantization Considerations

- TFLite strong support for 8-bit quantization; large size/speed gains
- Validate accuracy impact through representative calibration data
- Confirm microcontroller kernels exist for quantized path
- Prefer quantized activations/weights for devices without FPU





Practical Export Checklist

- Before training:
  - Export to TFLite; list unsupported ops
  - Decide app vs model boundary for pre/post
  - Choose target precision (INT8/FP16/FP32) consistent with device
- During training:
  - Maintain exportability; avoid training-only constructs in inference graph
- After training:
  - Re-export; verify size/speed; re-profile; update app-side code as needed



Project Generation vs Code Generation

- Project generation (not codegen): copy only required, unmodified sources
- Advantages:
  - Upgrades via standard merges (no generated monolith)
  - Multiple/replacement models without recompiling framework
  - Inline model data as C arrays (FlatBuffer used in-place; no parsing)
  - Self-contained, IDE-ready projects
- Control size: register only needed kernels via OpResolver

Generated Projects: Targets & Artifacts

Make pipeline emits projects for:	Mbed, Keil, Arduino, generic “raw” Make projects
Generated outputs include:	TensorFlow Lite Micro sources (only needed files) Third-party headers/libs (self-contained) IDE-specific adjustments where necessary

IDE-Specific Transformations

- Arduino:
  - Rename .cc → .cpp
  - Rewrite include paths to match Arduino constraints
- Generic Make/IDE:
  - Preserve folder hierarchy for easier merges back to source
- Note: Prebuilt generated archives may be published for convenience

Managing Binary Size with OpResolver

Strategy:

- Use minimal resolver: register only ops used by your model
- AllOpsResolver: convenient for prototyping, larger binary

Impact:

- Significant flash savings; fewer kernels linked

Tie resolver setup to model op list for deterministic builds

Specialization via Subfolders



STM32F746 Discovery board
Image source: digikey.hk

- Mechanism: target-named subfolders override default .cc implementations
  - Example: disco_f746ng/audio_provider.cc (real mic) vs default zeros
  - Same API surface; specialized internals per platform
- Also for generic speedups: portable_optimized/ kernels (not chip-specific)

Bring-Up: Minimal Required Platform Work

- Only mandatory platform function first: DebugLog(const char*)
- Validate with micro_error_reporter_test:
  - Expect success sentinel: “~~~ALL TESTS PASSED~~~”
- Then specialize I/O modules (e.g., audio_provider.cc) for real data
- Keep inference path allocation-free (fixed arena)

App–Model Boundary: A Practical Pattern

- Model:
  - Pure NN layers; supported ops; quantized where possible
  - No sensor I/O; no file/OS dependencies
- App:
  - Input: capture, decode/resize, FFT/mel (streaming cache)
  - Output: smoothing, thresholds, NMS, user feedback
  - Concurrency: threading for capture/prep (if platform supports)
- Outcome: smaller model; faster; easier debugging and portability

Debugging Export Failures

Symptoms:

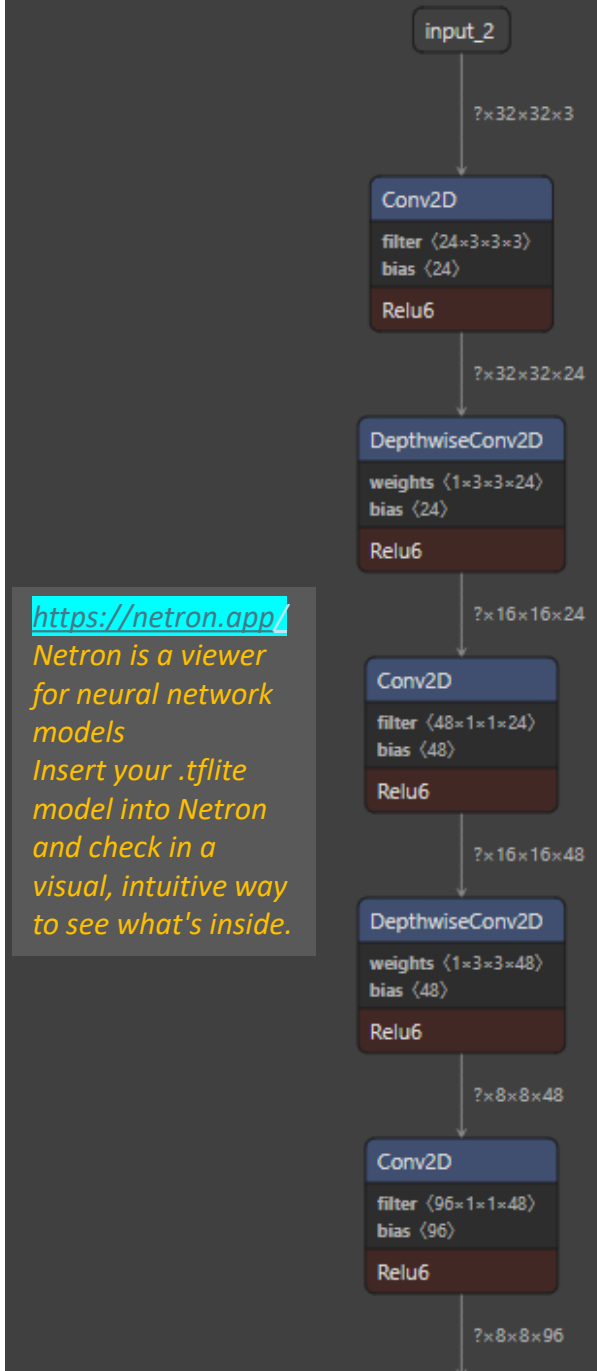
- Converter errors: unsupported ops; type mismatches; missing quantization
- Runtime errors: mismatched tensor shapes; op not registered

Triage:

- Netron: inspect TFLite graph; identify non-core preprocessing/post ops
- Replace or move ops to app code
- Re-export with allow_custom_ops if absolutely necessary

Iterate fast with small test graphs

https://netron.app/
Netron is a viewer for neural network models
Insert your .tflite model into Netron and check in a visual, intuitive way to see what's inside.



Measuring What Matters Early

- Measure on target (or close emulation) before full training:
- Inference latency; peak/arena memory; flash size
  - Use representative input pipelines (real sensor feed if possible)
- Validate quantized accuracy against a subset
- Reserve time for kernel-level optimization if needed



Case Pattern: Speech Commands



Mel features are audio descriptors that map a short-time spectrum onto the mel scale, which approximates how humans perceive pitch. Commonly you'll use log-mel spectrograms or MFCCs, giving a compact, noise-robust representation for speech models like wake-word detectors.

- Preprocessing:
  - Streaming audio; cache FFT intermediates; mel features app-side
- Core NN:
  - Convs/activations; quantized for MCU performance
- Postprocessing:
  - Averaging/smoothing over time for robust wake-word detection
- Benefit:
  - Lower compute; reduced memory; improved user-perceived accuracy

Case Pattern: Vision on Fixed Camera



- Preprocessing:
  - Fixed-size sensor; direct resize/crop in app; no JPEG decode on-device
- Core NN:
  - Mobile-class conv net truncated/quantized for MCU
- Postprocessing:
  - Thresholds; region-of-interest logic; optional light-weight NMS app-side
- Benefit:
  - Fewer graph ops; lower latency; simpler pipeline

Workflow Summary: TF → TFLite → TFLM

1. Define minimal core NN; avoid unsupported ops
2. Export early; list ops; iterate design
3. Partition pre/post to app code; plan quantization
4. Generate project; set minimal OpResolver; implement DebugLog
5. Specialize I/O; profile; optimize hot kernels if needed
6. Validate memory/latency; finalize deployment

Quick Checklist

01

Export early; list ops; remove training-only constructs

02

Push pre/post to app; exploit streaming/coherency

03

Prefer quantized ops; verify micro kernels exist

04

Register only necessary ops; implement DebugLog first

05

Specialize I/O via subfolders; consider portable_optimized kernels

06

Test for “~~~ALL TESTS PASSED~~~”; profile on target; iterate

Quiz 2 – Question 1

Best time to check TFLite op compatibility is:

A) After full training

B) Early, by exporting and listing ops

C) Only after deployment

D) Never

Quiz 2 – Answer 1

Best time to check TFLite op compatibility is:

A) After full training

B) Early, by exporting and listing ops

C) Only after deployment

D) Never

Export early to reveal unsupported ops and adjust design

Quiz 2 – Question 2

Recommended for preprocessing/postprocessing:

A) Implement in application code for efficiency and control

B) Keep all inside the TFLite graph

C) Disable preprocessing

D) Use training ops at inference

Quiz 2 – Answer 2

Recommended for preprocessing/postprocessing:

A) Implement in application code for efficiency and control

B) Keep all inside the TFLite graph

C) Disable preprocessing

D) Use training ops at inference

Move non-core steps to app code (e.g., JPEG decode, FFT, NMS)

Quiz 2 – Question 3

Unsupported ops can be handled by:

A) Ignoring errors

B) Arbitrary op substitution

C) Using training ops at inference

D) allow_custom_ops + AddCustom() kernel + FlexBuffers params

Quiz 2 – Answer 3

Unsupported ops can be handled by:

A) Ignoring errors

B) Arbitrary op substitution

C) Using training ops at inference

D) allow_custom_ops + AddCustom() kernel + FlexBuffers params

Save as custom ops, implement kernels, parse FlexBuffers

Quiz 2 – Question 4

Project generation (vs codegen) advantage:

A) Harder upgrades

B) Single-model restriction

C) Unmodified sources for easy merges; interpreter supports multiple/replacement models

D) Mandatory full recompilation on model change

Quiz 2 – Answer 4

Project generation (vs codegen) advantage:

A) Harder upgrades

B) Single-model restriction

C) Unmodified sources for easy merges; interpreter supports multiple/replacement models

D) Mandatory full recompilation on model change

Upgradability and flexibility without codegen downsides

Quiz 2 – Question 5

OpResolver in TFLite for Microcontrollers is used to:

A) Register all kernels always

B) Enable dynamic allocation

C) Replace FlatBuffers

D) Register only needed kernels to cut code size

Quiz 2 – Answer 5

OpResolver in TFLite for Microcontrollers is used to:

A) Register all kernels always

B) Enable dynamic allocation

C) Replace FlatBuffers

D) Register only needed kernels to cut code size

Keep binary minimal by registering only required ops

Quiz 2 – Question 6

Micro test success is detected via:

A) GUI dialog

B) Exit code only

C) “~~~ALL TESTS PASSED~~~” in logs

D) CSV output

Quiz 2 – Amswer 6

Micro test success is detected via:

A) GUI dialog

B) Exit code only

C) “~~~ALL TESTS PASSED~~~” in logs

D) CSV output

The framework prints a success sentinel in debug logs

Quiz 2 – Question 7

For platform bring-up, the only required function to implement first is:

A) Audio capture

B) File I/O

C) DebugLog(const char*)

D) GPU driver

Quiz 2 – Answer 7

For platform bring-up, the only required function to implement first is:

A) Audio capture

B) File I/O

C) DebugLog(const char*)

D) GPU driver

DebugLog enables visibility for tests and debugging

Quiz 2 – Question 8

Generated projects may need IDE-specific transforms such as:

A) YAML to JSON

B) .cc → .cpp and include path rewrites (e.g., Arduino)

C) Switching to Python

D) Removing headers

Quiz 2 – Answer 8

Generated projects may need IDE-specific transforms such as:

A) YAML to JSON

B) .cc → .cpp and include path rewrites (e.g., Arduino)

C) Switching to Python

D) Removing headers

The build system adapts files for specific IDEs

Quiz 2 – Question 9

TFLite export removes:

A) Training/backprop ops

B) Inference ops

C) All convolutions

D) All activations

Quiz 2 – Answer 9

TFLite export removes:

A) Training/backprop ops

B) Inference ops

C) All convolutions

D) All activations

Training-specific operations are stripped

Quiz 2 – Question 10

Netron is useful for:

A) Compiling kernels

B) Measuring power

C) Replacing the interpreter

D) Inspecting TFLite graphs to isolate the core network

Quiz 2 – Answer 10

Netron is useful for:

A) Compiling kernels

B) Measuring power

C) Replacing the interpreter

D) Inspecting TFLite graphs to isolate the core network

Visualize ops, separate core NN from pre/postprocessing



SESSION 3: AUTOMATED OPTIMIZATION SYSTEMS

Modern AI Compiler Architecture

XLA, TVM, and MLIR Approaches

XLA (Accelerated Linear Algebra):
Applies graph-level transformations including fusion and layout optimizations for TPUs and GPUs. Generates optimized code for specific hardware targets with automatic kernel selection.

TVM (Tensor Virtual Machine):
Provides cross-platform optimization with automated performance tuning. AutoTVM explores different kernel implementations and selects optimal strategies for each hardware target.

MLIR (Multi-Level Intermediate Representation):
Enables flexible multi-stage transformations at different abstraction levels. Supports custom dialects for domain-specific optimizations while maintaining interoperability.

These systems demonstrate convergent evolution toward automated optimization that matches or exceeds manual optimization quality while providing framework flexibility and hardware portability.

TFLM Design Goals & Constraints

- No OS deps; runs on bare-metal/minimal RTOS
- No standard C/C++ library at link (except math)
- No FPU expected; prioritize INT8 inference
- No dynamic allocation during inference (fixed arena)
- C++11; 32-bit targets
- Outcome: predictable latency, small portable binaries

Why Interpret Models vs Codegen

Interpreted graph: static code + FlatBuffer model data

Pros: easy upgrades (unmodified sources), swap/multiple models without rebuilding framework, inline model as C array (zero-copy)

Project generation provides codegen-like convenience without monolithic generated code

Project Generation & Build Outputs

Project generation is the process of creating a ready-to-build firmware project that includes only the TensorFlow Lite for Microcontrollers sources you need, plus minimal helpers and third-party headers, arranged for your target board or IDE. It doesn't generate model code; it simply curates unmodified TFLM sources so upgrades stay easy and binaries stay small. The "tool" for this, per Chapter 13, is TFLM's Makefile-based build system under tensorflow/lite/micro/tools/make, which you run with make to build tests, cross-compile for a board, or emit self-contained projects for environments like Mbed, Keil, or Arduino.

- Copies only needed TFLM sources (unmodified)
- Targets: Mbed, Keil, Arduino, generic Make
- Outputs: target-specific source tree, IDE files, bundled deps
- Note: prebuilt generated archives may be provided

Managing Code Size with OpResolver

Minimal resolver: register only required kernels
AllOpsResolver: convenient for prototyping; larger binary
Tie resolver to model’s op list (from conversion/inspection)
Result: deterministic, smaller flash footprint

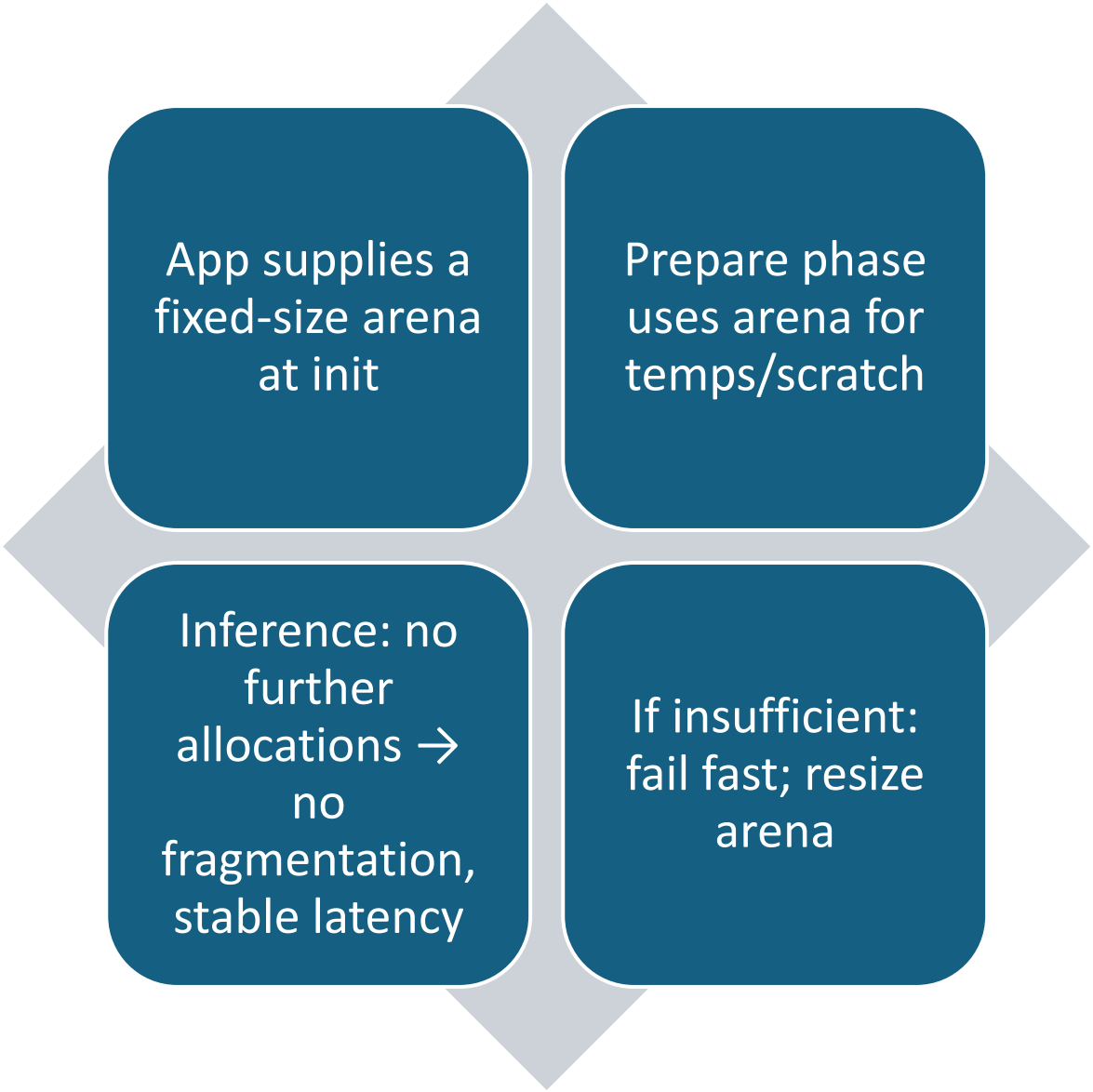
Specialization via Subfolders

- Target-named subfolders override defaults (same API)
- Example: disco_f746ng/audio_provider.cc for real mic capture
- portable_optimized/ kernels: generic speedups (not chip-specific)

Build Mechanics: Targets and Tags

- Makefile specialize() selects platform/feature sources
- ALL_TAGS includes target (e.g., sparkfun_edge) + optional tags (e.g., portable_optimized)
- Platform .inc files define toolchain/flags/downloads (e.g., apollo3evb_makefile.inc)

Memory Model: Fixed Arena



FlatBuffers Essentials

Model root: version, buffers (const data), subgraphs

Tensors: shape, type, buffer index, name, quantization, is_variable

Operators (topological order): opcode_index, inputs/outputs, builtin_options or custom_options (FlexBuffers)

Zero-copy: tflite::GetModel(ptr); check schema version

Built-in vs Custom Ops

- Built-ins: params via builtin_options (schema-defined)
- Custom: params via FlexBuffers (map)
- Execution order already topological; no runtime graph analysis
- Micro focuses on subset of built-ins; customs supported when required

Bring-Up & Micro Tests

- First implement DebugLog(const char*)
- Run micro_error_reporter_test; expect “~~~ALL TESTS PASSED~~~”
- Micro test framework: TF_LITE_MICRO_TESTS_BEGIN/END, TF_LITE_MICRO_EXPECT_*; no dynamic memory/external deps

Desktop-to-Device Flow & IDE Adapters

- Iterate on x86 (Bazel/Make) → cross-compile Make TARGET=<board>
- Flash and monitor DebugLog
- IDE transforms: Arduino .cc→.cpp, include path rewrites; preserve hierarchy elsewhere
- Keep projects self-contained

Porting Mobile Ops to Micro

1. Create micro kernel (tensorflow/lite/micro/kernels/<op>.cc)
2. Remove dynamic allocation/STL/external deps; keep reference_ops path in Eval()
3. Port tests to Micro framework; stack-based data; single-file tests
4. Use AllOpsResolver for convenience in early testing if needed

Accelerator Integration Model

- Synchronous coprocessor approach
- Replace expensive kernels with accelerator calls
- Keep data CPU-addressable; no async subgraphs in TFLM
- Provide fallback to reference path for portability

Practical Patterns & Diagnostics

Speech: app-side FFT/mel, quantized convs, smoothing; optimize depthwise_conv via portable_optimized

Vision: app-side resize/crop, small quantized conv net, lightweight postproc

Diagnostics: instrument DebugLog for arena usage and inference timing; size arena conservatively then trim

Pitfalls, Checklist, and Summary



Pitfalls: linking stdlib (bloats binary), hidden allocations, broad resolver, coupling I/O into model



Checklist: minimal resolver; DebugLog first; fixed arena; topological ops; custom options via FlexBuffers; test “ALL TESTS PASSED”



Summary: constrained-by-design TFLM + project generation + specialization + fixed arena → small, reliable MCU deployments; accelerators via kernel swaps

Summary - Week 8 Key Takeaways

Automated ML Compilation Mastery

Compilation Pipeline Understanding:
ML compilation transforms high-level computation graphs through systematic stages: graph optimization, kernel selection, memory planning, computation scheduling, and code generation.

Cross-Platform Deployment:
Successful deployment requires careful separation of core neural network computation from preprocessing/postprocessing, with automated project generation enabling platform-specific optimizations.

Framework Design Principles:
Resource-constrained environments drive design decisions toward simplicity, determinism, and portability while maintaining optimization opportunities through specialization mechanisms.

Practical Implementation:
Modern ML frameworks automate complex optimization decisions while providing escape hatches for platform-specific customization when needed.

Hardware Integration:
Custom accelerators integrate most effectively through kernel-level replacement within established frameworks rather than building entirely new software stacks.

Looking Ahead - Week 9 Preview

Pruning and Sparsity: Optimizing Models Through Strategic Parameter Removal

Learning Objectives:

- Master pruning techniques for reducing model complexity while preserving accuracy
- Analyze the trade-offs between unstructured, structured, and dynamic pruning approaches
- Design pruning strategies for different deployment scenarios and constraints
- Implement practical pruning workflows using modern frameworks

Required Readings:

- Book 1 – Volume 1: Model Compression
- Book 1 – Volume 1: Hardware Acceleration
- Book 1 – Volume 1: Benchmarking
- Book 1 – Volume 1: Model Serving
- Book 2: Chapter 15: Optimizing Latency
- Book 2: Chapter 17: Optimizing Model and Binary Size